AI VIET NAM | All-in-One Course 2026

# Solution
# Kiến trúc Apache Kafka

Đáp án và giải thích — Kỹ thuật dữ liệu

**Bảng đáp án**

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| C | A | D | B | C | B | C | A | B | D |

## Giải thích đáp án

### Câu 1. Đáp án C — Vai trò producer, broker và consumer

*Producer gửi record; broker lưu và phục vụ dữ liệu; consumer đọc và thực hiện xử lý.*

Producer công bố dữ liệu, broker lưu các replica và phục vụ yêu cầu từ client, còn consumer đọc để xử lý. Controller quản lý metadata của cluster; nó không phải trạm trung chuyển bắt buộc cho mọi lần ghi hoặc thành phần thực hiện nghiệp vụ tồn kho.

### Câu 2. Đáp án A — Thứ tự và phạm vi offset

*Mỗi partition có thứ tự log riêng; offset 7 ở P0 và offset 7 ở P1 là hai vị trí khác nhau.*

Offset được xác định trong từng partition. Kafka duy trì thứ tự của log theo partition, không tạo một thứ tự toàn cục cho topic nhiều partition. Thứ tự hoàn tất xử lý còn phụ thuộc ứng dụng, nhất là khi consumer xử lý song song.

### Câu 3. Đáp án D — Key và phân vùng

*Được đưa vào cùng một partition theo chiến lược phân vùng đã nêu.*

Trong các điều kiện của câu hỏi, cùng key được ánh xạ về cùng partition. Điều đó hỗ trợ theo dõi vòng đời đơn hàng trong một log. Thay đổi số partition hoặc chiến lược phân vùng có thể làm ánh xạ thay đổi; key cũng không nhất thiết là mã sự kiện duy nhất.

**Câu 4. Đáp án B — Record không có key**

*Producer có cơ chế sticky gắn với batching; không nên giả định round robin cho từng record.*

Java producer mặc định sử dụng cách chọn partition dạng sticky cho record không key, gắn với việc gom batch. Vì vậy, quan sát một lượng nhỏ record không đủ để yêu cầu số lượng bằng nhau ở từng partition. Client khác hoặc partitioner tùy chỉnh có thể có hành vi khác.

**Câu 5. Đáp án C — Partition và replica**

*Có 3 partition logic và 9 replica tổng cộng, trong đó có 3 leader.*

Replication factor tính cả leader. Mỗi partition có một leader và hai follower, nên ba partition có tổng cộng 3 nhân 3 bằng 9 replica. Tạo bản sao không làm tăng số partition logic của topic.

**Câu 6. Đáp án B — ISR và xác nhận ghi**

*Ngưỡng 2 kiểm tra ISR tối thiểu; acks=all chờ ISR hiện tại, không chỉ hai replica bất kỳ.*

min.insync.replicas là ngưỡng tối thiểu của ISR để nhận ghi với acks=all trong mô hình đã nêu. Nó không đổi replication factor và không giới hạn việc xác nhận ở đúng hai replica. Với ISR hiện có ba replica, acks=all chờ ISR đó; xác nhận ghi cũng khác với hoàn tất xử lý ở consumer.

**Câu 7. Đáp án C — Chẩn đoán lag lệch theo partition**

*Đo thời gian xử lý và kiểm tra hệ thống đích của consumer P1; xem lại tải của key nóng trước khi đổi cách phân vùng.*

Lag tập trung ở P1; cần kiểm tra tốc độ vào, tốc độ xử lý và hệ thống đích của partition này. A nhầm cơ chế phân công của group thông thường. B sao chép log nhưng không chia việc đọc trong group. D có thể dàn tải nhưng bỏ ràng buộc thứ tự theo key.

**Câu 8. Đáp án A — Bố trí consumer group cho hai pipeline**

*Dùng hai instance cùng group `stock`; báo cáo dùng group khác và chọn vị trí replay riêng của báo cáo.*

Hai instance tồn kho cùng group để chia partition; báo cáo có group riêng để đọc đủ luồng và quản lý mốc replay riêng. B chia bản ghi giữa cả hai loại tác vụ. C khiến mỗi instance tồn kho đọc luồng độc lập. D nhầm vai trò của replica với committed offset theo group.

**Câu 9. Đáp án B — Chẩn đoán ghi lỗi khi replica tụt lại**

*P2 không đáp ứng ngưỡng ISR; khôi phục ít nhất một follower và xác nhận ISR trở lại từ 2 thành viên trước khi kỳ vọng lần ghi thành công.*

Leader còn hoạt động chưa đủ: P2 chỉ có một thành viên ISR, dưới ngưỡng 2. B khôi phục replica theo kịp thay vì đổi điều kiện xác nhận. A nhầm số bản sao cấu hình với số replica trong ISR. C nhầm xác nhận ghi với tiến độ đọc. D hạ chính ngưỡng cần giữ.

**Câu 10. Đáp án D — Khôi phục sau lỗi giữa ghi dữ liệu và commit offset**

*Đọc lại từ mốc 41, ghi dấu `event_id` và cập nhật tồn kho trong cùng giao dịch khi xử lý mới; bỏ qua tác động đã ghi nhận rồi commit đến vị trí tiếp theo sau khi xử lý xong.*

Mốc đã commit vẫn là 41, nên có thể đọc lại cả 41 và 42. D dùng một giao dịch để ràng buộc dấu đã xử lý với cập nhật tồn kho, rồi mới commit tiến độ. A và C có thể bỏ sót 42 vì fetch không đồng nghĩa xử lý xong. B không bảo vệ tác động trong cơ sở dữ liệu ngoài Kafka.